SoftwareX 34 (2026) 102546

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

SurVigilance: An application for accessing global pharmacovigilance data

Raktim Mukhopadhyay [a, b, *], Marianthi Markatou [a, c]

[a] Department of Biostatistics, University at Buffalo, 726 Kimball Tower, 3435 Main Street, Buffalo, NY, 14214, USA
[b] Institute for Artificial Intelligence and Data Science, University at Buffalo, Buffalo, NY, USA
[c] Department of Medicine, Jacobs School of Medicine and Biomedical Sciences, University at Buffalo, Buffalo, NY, USA

ARTICLE INFO

Keywords:
Pharmacovigilance
Spontaneous reporting systems
Adverse events
Safety database
SeleniumBase

ABSTRACT

Even though several publicly accessible pharmacovigilance databases are available, extracting data from them is a technically challenging process. Existing tools typically focus on a single database. We present SurVigilance, an open-source tool that streamlines the process of retrieving safety data from seven major pharmacovigilance databases. SurVigilance provides a graphical user interface as well as functions for programmatic access, thus enabling integration into existing research workflows. SurVigilance utilizes a modular architecture to provide access to the heterogeneous sources. By reducing the technical barriers to accessing safety data, SurVigilance aims to facilitate pharmacovigilance research.

Metadata

Code metadata.

Nr.	Code metadata description	Metadata
C1	Current code version	1.0.1
C2	Permanent link to code/repository used for this code version	https://github.com/rmj3197/SurVigilance
C3	Permanent link to Reproducible Capsule	https://codeocean.com/capsule/7678655/tree/v1
C4	Legal Code License	GNU General Public License (GPL-3.0)
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python, Streamlit, SeleniumBase
C7	Compilation requirements, operating environments & dependencies	Supports multiple operating systems; Needs installed Google Chrome web browser; Internet access is required for application to access the databases; Python $\geq$ 3.10, $\leq$ 3.13, Python packages: beautifulsoup4, lxml, openpyxl, pandas, requests, seleniumbase, streamlit
C8	If available, link to developer documentation/manual	https://survigilance.netlify.app/
C9	Support email for questions	raktimmu@buffalo.edu

1. Introduction

According to the World Health Organization (WHO), "first, do no harm" is one of the most fundamental principles of healthcare [1]. However, individuals experience adverse events (AEs) after the administration of drugs, vaccines, and other therapeutic biologics. This raises a serious public health concern, which is associated with increased costs [2], and results in hospitalizations and deaths [3]. Clinical trials provide the highest standard of evidence for safety information on drugs, vaccines, and other therapeutic biologics before a drug is licensed by regulatory authorities. However, clinical trials suffer from a number of limitations that affect their generalizability [4], and hence underscore the need for post-marketing surveillance.

The identification of safety issues related to a product after its introduction to the market is primarily conducted using the data obtained from spontaneous reporting systems (SRS), examples of which include the Food and Drug Administration (FDA) Adverse Event Reporting System (FAERS) and VigiBase, maintained by the WHO, among others.

* Corresponding author at: Department of Biostatistics, University at Buffalo, 726 Kimball Tower, 3435 Main Street, Buffalo, NY, 14214, USA.
Email addresses: raktimmu@buffalo.edu (R. Mukhopadhyay), markatou@buffalo.edu (M. Markatou).

https://doi.org/10.1016/j.softx.2026.102546
Received 13 October 2025; Received in revised form 8 December 2025; Accepted 28 January 2026
Available online 6 February 2026

SRS are passive surveillance systems that collect information on potential safety issues associated with the use of medical products submitted by healthcare workers, patients and their caregivers. For several decades, pharmaceutical corporations, public health, and regulatory authorities have relied heavily on SRS databases to conduct post-marketing surveillance of medications, vaccines, biologics and medical devices.

Several methods of disproportionality analysis have been developed and applied primarily to data derived from such safety databases. These methods include Proportional Reporting Ratio, Reporting Odds Ratio, Bayesian Confidence Propagation Neural Network, Multi-item Gamma Poisson Shrinker, Likelihood Ratio Test (LRT)-based methods, to name a few. A fairly complete description of the methods appears in [5], while various LRT-based methods can be found in [6,7], with TreeScan [8] being an alternative method. Regardless of the data representation, all of these methods require count data associated with the suspected AEs in order to be applied.

Let's assume that a contingency table contains I suspected AEs ($i = 1, 2, \ldots, I$) associated with J drugs ($j = 1, 2, \ldots, J$). An illustration of the 2×2 contingency table used in these methods is shown in Table 1. The count of the i-th AE and j-th drug combination is represented by n_{ij}. The marginal row total of the i-th AE is represented as $n_{i\cdot}$, while the column marginal of the j-th drug is $n_{\cdot j}$. Mathematically, $n_{i\cdot} = \sum_{j=1}^{J} n_{ij}$ and $n_{\cdot j} = \sum_{i=1}^{I} n_{ij}$. The total number of suspected AEs in the contingency table is $n_{\cdot\cdot}$. The authors in [5] introduced two different methods of contingency table construction, namely AE-based and drug-based comparisons. A brief discussion on this aspect is included in Section 3.2.2.

Table 1
Contingency table for AE-based signal detection.

	$Drug_j$	Other drugs	AE marginal total
AE_i	n_{ij}	$n_{i\cdot} - n_{ij}$	$n_{i\cdot}$
Other AEs	$n_{\cdot j} - n_{ij}$	$n_{\cdot\cdot} - n_{i\cdot} - n_{\cdot j} + n_{ij}$	$n_{\cdot\cdot} - n_{i\cdot}$
Drug marginal total	$n_{\cdot j}$	$n_{\cdot\cdot} - n_{\cdot j}$	$n_{\cdot\cdot}$

The accessibility of eleven open-access pharmacovigilance databases was investigated in 2016 [9]. Out of the eleven databases investigated in the study, our tool, `SurVigilance`, currently supports the seven databases described below:

- Australian Database of Adverse Event Notifications (DAEN): DAEN (medicines) contains AE data associated with medicines or vaccines. The database provides both aggregated data and detailed reports organized chronologically for a specific medicine or vaccine. The DAEN (medicines) has data dating back to January 1971, while the DAEN (medical products) includes reports starting from July 1, 2012 [10]. We currently support access to data from the DAEN (medicines) database in the tool. The database can be accessed at https://www.tga.gov.au/safety/database-adverse-event-notifications-daen.
- Danish Medicines Agency (Lægemiddelstyrelsen) Database (DMA): DMA provides aggregated data on suspected AEs reported in Denmark for drugs. The data is derived from the DMA pharmacovigilance database, which contains reports since 1968 [11]. The database can be accessed at https://laegemiddelstyrelsen.dk/en/sideeffects/side-effects-of-medicines/interactive-adverse-drug-reaction-overviews/.
- Netherlands Lareb Database: The Netherlands pharmacovigilance centre Lareb maintains a database of AEs for both drugs and vaccines. The database was established in 1995; since then, it has received close to 20,000–25,000 reports annually from patients, healthcare professionals, and pharmaceutical companies [12,13]. The database can be accessed at https://www.lareb.nl/en/.
- New Zealand Medsafe Database: The Medsafe database contains spontaneous reports submitted in New Zealand since 1965 [9]. The database provides access to both aggregated data and de-identified spontaneous reports for a drug or vaccine of interest. The database can be accessed at https://www.medsafe.govt.nz/SMARS/Default.
- United States Food and Drug Administration (FDA) Adverse Event Reporting System (FAERS): The FAERS database contains the spontaneous reports submitted to the FDA. FAERS provides access to the de-identified spontaneous reports submitted by patients, healthcare providers and pharmaceutical companies [14]. Quarterly updates of the database can be found at https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html.
- United States Vaccine Adverse Event Reporting System (VAERS): VAERS contains reports of AEs following immunization. This database is maintained by the Centers for Disease Control and Prevention (CDC) and FDA [15]. The VAERS database can be accessed at https://vaers.hhs.gov/data/datasets.html.
- WHO VigiAccess: VigiBase, maintained by the WHO, is the largest pharmacovigilance database in the world, with >30 million reports submitted since 1968 by member countries of the WHO Programme for International Drug Monitoring (PIDM) [16]. In 2015, the WHO launched VigiAccess to provide public access to aggregated data from VigiBase [17]. The database can be accessed at https://www.vigiaccess.org/.

The authors in [9] found that databases in the United States offered a high level of access, while the databases from New Zealand and Australia provided a medium level of access. The databases maintained by the WHO, the Netherlands, and Denmark provided comparatively limited access. United States databases such as FAERS and VAERS grant access to individual case safety reports (ICSRs) as well as allow full database downloads, which also facilitate the calculation of population denominators and represent a high degree of transparency. Australia and New Zealand provide access to certain demographic information such as age and drugs/vaccines contained in each ICSR, and hence are considered to provide a medium level of access. In contrast, the databases from the WHO, the Netherlands, and Denmark only provide aggregated data on AE counts per drug and summarized demographic information. Since these databases do not provide complete database downloads and access to all ICSRs, computing the population denominator is more complex. In addition to the databases supported by `SurVigilance`, other publicly available databases are European Medicines Agency (EMA) EudraVigilance, United Kingdom (UK) Yellow Card, and Canadian MedEffect databases. The EudraVigilance database uses a legacy system to provide access to data, while the UK database presents the data in a highly complex format. As a result, accessing data from these sources is significantly more challenging than others and hence is not included in `SurVigilance`. The MedEffect database is available as a single download from https://www.canada.ca/en/health-canada/services/drugs-health-products/medeffect-canada/adverse-reaction-database/canada-vigilance-online-database-data-extract.html, and hence has not been included in the tool due to the simplicity of accessing the data.

In this paper, we present `SurVigilance`, a tool that allows users to access data from multiple safety databases across the world. The primary goal of this tool is to facilitate pharmacovigilance researchers' access to safety data. Many existing safety databases restrict users by allowing

them only to view and not download the data (e.g., WHO VigiAccess). SurVigilance addresses this limitation by enabling researchers to download safety data. SurVigilance reduces the complexity of navigating different databases while streamlining the process of data retrieval for further analysis. The tool supports two complementary modes of use: first, a simple and user-friendly graphical user interface that removes the barrier for non-technical users; and second, a set of functions that provide programmatic access to the same data retrieval functionalities. The multiple access methods allow SurVigilance to be integrated into workflows and larger data pipelines, making it suitable for both exploratory analysis and reproducible research. We hope that this tool aids the scientific community in the investigation of safety concerns associated with various medical products.

2. Software description

SurVigilance is implemented using the Streamlit framework in the Python programming language. Since SurVigilance is implemented using Streamlit, the application requires only a web browser to run, eliminating complications associated with additional installers and hardware-dependent customizations (e.g., arm64- or x86_64-specific installers). SurVigilance provides access to seven databases: DAEN, DMA, Lareb, Medsafe, FAERS, VAERS, and VigiAccess. In order to provide access to data from these databases, SurVigilance uses a modular approach, where the code for each safety database is kept independent of the others. This enhances the readability of the code and ensures that the codebase is maintainable and extensible for future development. A schematic diagram highlighting the various functionalities and the process of accessing the various databases is depicted in Fig. 1. Salient features of the software are highlighted below:

- User Interface: As noted previously, an easy-to-use user interface is created using the Streamlit framework. The UI class of the ui module provides the graphical interface as shown in Fig. 1. The _app.py file provides the home page of the application. The home page provides users the options to select the location where data files will be saved and to select a database from which data will be accessed. Upon selecting the relevant database, the user is then taken to the respective page dedicated to the database. Each database has a dedicated page located under ui/pages that provides the graphical interface. These pages include the relevant calls to functions and customizations needed for downloading the data from the different safety databases. For the DAEN, DMA, Medsafe, Lareb, and VigiAccess databases, we provide the option to search for specific drugs or vaccines for which data needs to be downloaded. In contrast, for the FAERS and VAERS databases, we list all the years for which the data is available and download the entire database for that period. This difference stems from the manner in which each safety database permits access to its underlying data.
- Scrapers and Programmatic Access: The code that enables SurVigilance to scrape or download the data lives in the scrapers/ submodule within ui/, depicted in Fig. 1. The various functions in this submodule enable programmatic access to the data retrieval methods without the need for graphical interfaces. An overview of how the various scrapers work is also included in Fig. 1. A discussion on the web-automation/scraping is included in Section 2.1.
- Data Storage: The application allows users to specify a location where the downloaded data will persist even after the application is closed. A sub-folder for each database is created inside the location, in which the downloaded data is stored. The data from FAERS and VAERS databases are stored in a compressed file format (.zip), as they include export of the entire database comprising of multiple files, and made available in that format on the FAERS and VAERS webpages. The data from DAEN is stored as .xlsx format as the DAEN webpage exports the data in that format. Data from all other databases are stored as .csv after extracting the data from their respective webpages. The other databases do allow to create exports of the data or download the data.

2.1. Web-automation/scraping

The web-automation, scraping and downloading functionality of the tool is built using SeleniumBase [18]. SeleniumBase is a browser automation framework built on top of Selenium. Our choice to use SeleniumBase is associated with a number of advantages. Browser automation generally requires a WebDriver[1] that is compatible with the installed web browser. SeleniumBase manages its own drivers, thereby eliminating the complexity for users of providing access to an appropriate WebDriver. SeleniumBase offers two different ways of interacting with the browser, namely the undetectedchromedriver (UC) and Chrome DevTools Protocol (CDP) modes.[2] These options allow robust web automation needed for retrieving the data. Please see Appendix A for a discussion on the various measures employed for responsible web scraping.

We provide a function (check_site_connectivity) to check the connectivity to the supported databases before initiating the data retrieval process. The various webpages associated with the databases allow access to data only after performing a set of actions in a specific sequence, and hence a primary challenge was handling interactions with various web elements present on a webpage. To ensure reproducible data extraction, we intentionally added wait times at various stages of the scraping process. This ensures that each required web element is fully loaded and behaves as expected before we attempt to interact with it in order to retrieve the data. In addition to the wait times, we include a retry mechanism in SurVigilance that retries the data collection if the process fails for any reason. We accomplish this using the num_retries parameter which specifies the number of times the data collection should be retried before showing an error to the user. We set num_retries to a default value of 5, and provide the ability to set custom values both in the GUI as well as in the data retrieval functions associated with the various databases. The FAERS and VAERS provide access to the data files through dedicated endpoints and can be downloaded simply using the requests module, while the other databases rely on dynamically rendered web components that appear only after certain actions are performed. The extensive use of dynamic web elements complicates the extraction of data from the Document Object Model (DOM). This necessitated careful analysis of the structure of each webpage using browser developer tools, specifically Chrome DevTools. We also note that, for the VAERS database, users must complete a verification CAPTCHA when downloading data.

2.2. Related software

Although we were not able to find software that scrapes or downloads data from multiple safety databases, we found software that either

[1] WebDrivers are open-source tools that provide a programmatic interface to interact with a browser in order to accomplish various web-automation tasks. Details can be found in [19].

[2] Please see [20] for a detailed explanation of WebDriver and CDP.

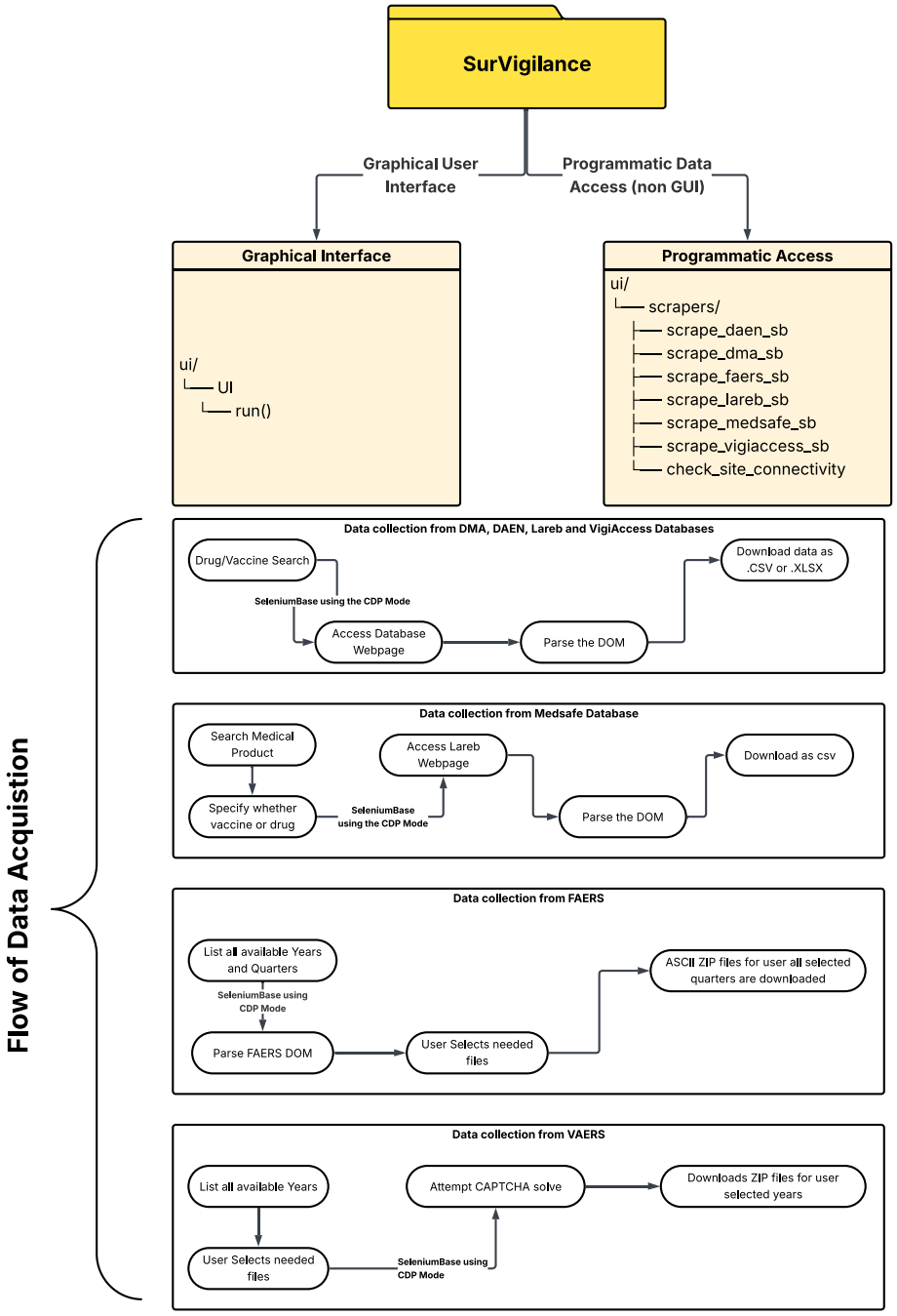

Fig. 1. A simplified overview of the main functionalities of `SurVigilance` is depicted. `SurVigilance` provides the graphical interface using the `UI` class of the `ui` module. The scraping functions implemented in `scrapers` sub-module of the `ui` module can also be accessed separately to perform data collection in a programmatic manner. Additionally, the process of data acquisition from each database is illustrated. The abbreviation DOM stands for Document Object Model, which represents the structure of a webpage.

Table 2
Comparison of pharmacovigilance software packages including access methods and analysis capabilities. Active development is determined by new versions between Jan 1, 2025, and Dec 4, 2025.

Packages/software	Brief description	Programming language	Type of data access	Downstream analysis/task	Active development	Repository	Unit tests (Coverage %)	Continuous integration
FAERS-focused tools								
`faers`	Access and analyze FAERS data	`R`	Quarterly data extract API	Disproportionality analysis	Yes	`Bioconductor`	✓ (27.30)	✓
`extractFAERS`	Process FAERS data	`R`	✗	Data Pre-processing	Yes	`CRAN`	✗	✗
`faersquarterlydata`	Process FAERS data	`R`	Quarterly data extract API	Data pre-processing and dispro-portionality analysis	Yes	`CRAN`	✓ (99.22)	✗
`FAERS`	Download and pre-process FAERS data	Bash	✗⋆	Creating SQL database with FAERS data	No	`GitHub`	✗	✗
`OpenVigil FDA`	Access and analyze FAERS data	`Java, PHP`	openFDA API	Disproportionality	No	`SourceForge`	✗	✗
VAERS-Focused Tools								
`vaers`	Snapshot of VAERS data	`R`	Snapshot of data as RData	✗	No	`GitLab`	✗	✗
`vaersvax`	Snapshot of VAERS data	`R`	Snapshot of data as RData	✗	No	`GitLab/CRAN`	✗	✗
`vaersNDvax`	Snapshot of VAERS data	`R`	Snapshot of data as RData	✗	No	`GitLab/CRAN`	✗	✗
Other SRS-Focused Tools								
`vigicaen`	Process VigiBase data	`R`	✗	Various data processing and disproportionality analysis functions	Yes	`CRAN`	✓ (92.03)	✓
Comprehensive Multi-Database Tools								
`SurVigilance`	Access to multiple safety databases. Supported databases: FAERS, VAERS, VigiAccess, Lareb, Medsafe, DAEN, DMA	`Python`	Quarterly data extract API for FAERS data, Web scrap-ing for other databases	✗	Yes	`GitHub/PyPI`	✓ (92.15)	✓

⋆: The API endpoints used in this work are obsolete at this moment

provides access to a single database or processes data. A large number of software packages are available to either download or process data from the FAERS database. The `faers` package [21] accesses the same API endpoints used by `SurVigilance` to download `FAERS` data. The `extractFAERS` [22] and `faersquarterlydata` [23] packages also provide various functions to process the data that has been downloaded from the FAERS database. The `FAERS` package [24] provides code to set up a `SQL` database using the `FAERS` data. The `OpenVigil` software [25] is a set of open-source software packages (`OpenVigil 1, 2`, and `OpenVigilFDA`) designed for the analysis of pharmacovigilance data from SRS. It provides tools for data mining, such as disproportionality analysis methods. `OpenVigil FDA` [26] accesses data using the `openFDA` interface provided by FAERS to access and analyze data. `OpenVigil 1` and `2` provide methods to work with data from other sources such as DrugBank data. For the VAERS database, three packages `vaers` [27], `vaersvax` [28] and `vaersNDvax` [29] provide snapshots of the data. The `vigicaen` package [30] provides various functionalities to process and analyze data from `VigiBase`. In Table 2, we present a brief comparison of the packages in terms of various implementation aspects such as programming language, active development, package repository, and coverage. This list further illustrates the scarcity of software tools that provide access to data from databases other than FAERS and VAERS, along with a complete absence of any software that provides access to multiple databases simultaneously, hence underscoring the need for `SurVigilance`.

Apart from these packages, the authors in [31] have provided a non-exhaustive list of software that provides methods for signal detection.

3. Illustrative examples

3.1. Programmatic data retrieval from the DMA database

In this example, we demonstrate the use of functions available in the `scrapers` module to extract data from the DMA database for the class of alpha-blocker drugs. We specifically consider five drugs: Alfuzosin, Doxazosin, Prazosin, Tamsulosin, and Terazosin. The code to access the data is presented in Listing 1. Furthermore, we create a contingency table with these five drugs, as shown in Table 3.

Note that the “Other Drugs” column is not present in the contingency table presented in Table 3. Since databases such as DMA do not allow full database downloads and instead restrict users to only search for specific medical products, a predefined list of drugs is needed to create the “Other Drugs” column. The composition of the drugs in that list depends on the kind of analysis the researcher is interested in performing. A more detailed discussion on the construction of the “Other Drugs” column is presented in the next example, in Section 3.2.2.

```python
import os
import pandas as pd

# Import the scraper function to get data from the DMA safety database
from SurVigilance.ui.scrapers import scrape_dma_sb


# List of drugs
DRUGS = ["Alfuzosin", "Doxazosin", "Prazosin", "Tamsulosin", "Terazosin"]

def get_dma_data():
    out_dir = "dma_out"
    os.makedirs(out_dir, exist_ok=True)

    all_results = []
    # Scrape data for each drug and collect results

    for drug in DRUGS:
        df = scrape_dma_sb(medicine=drug, output_dir=out_dir, headless=True)
        df = df.copy()
        df["Drug"] = drug
        df["Count"] = pd.to_numeric(df["Count"], errors="coerce").fillna(0)
        all_results.append(df)


    combined = pd.concat(all_results, ignore_index=True)


    # Build a contingency table
    contingency = combined.pivot_table(
        index="PT", columns="Drug", values="Count", aggfunc="sum", fill_value=0
    )


    # Save the contingency table to a CSV file
    contingency_path = os.path.join(out_dir, "dma_contingency_table.csv")
    contingency.to_csv(contingency_path)

    return contingency


if __name__ == "__main__":
    get_dma_data()
```

Listing 1. Python code demonstrating the use of functions to access data in a programmatic manner without the need for a graphical interface.

Table 3
A small portion of the contingency table created using the data for the five alpha-blocker drugs extracted from the DMA database.

PT[a]	Alfuzosin	Doxazosin	Prazosin	Tamsulosin	Terazosin
Dizziness	32	9	3	23	2
Syncope	11	5	7	6	1
Fatigue	10	6	8	5	2
Headache	9	10	10	7	1

[a]Please note that PT is an abbreviation for Preferred Term. PTs are used to represent specific AEs or symptoms. A detailed definition of PT can be found in [32].

3.2. FAERS data download and processing

In this section, we present an example that begins with downloading data from the FAERS database using SurVigilance, followed by the construction of a contingency table using the retrieved data.

3.2.1. Data download from FAERS

In order to download the data, the user needs to first instantiate the application as shown in Listing 2.

This opens the landing page (Figs. 2 and 3). On the landing page, when the user clicks on the FAERS button, the application navigates to the corresponding "Search Page for FAERS Database" (Fig. 4). On this page, the user first clicks on the "List all FAERS years and available quarters" button, which generates a list of all available quarters for the

```
>>> from SurVigilance.ui import UI
>>> UI().run()

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8502
  Network URL: http://192.168.12.140:8502
```

Listing 2. Python code for instantiating SurVigilance.

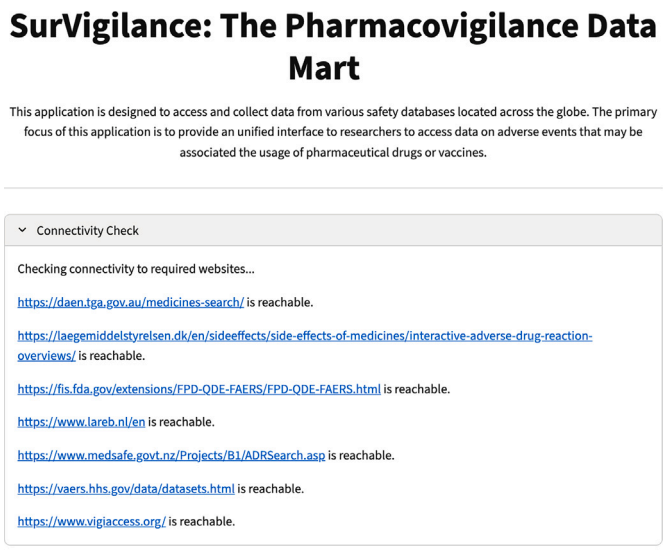

Fig. 2. The home page of SurVigilance contains an expandable element to monitor the connection status to the various databases. Every time the home page is refreshed, the application runs the connectivity check and reports the most updated connection status. This is accomplished by using the check_site_connectivity function in the scrapers module.

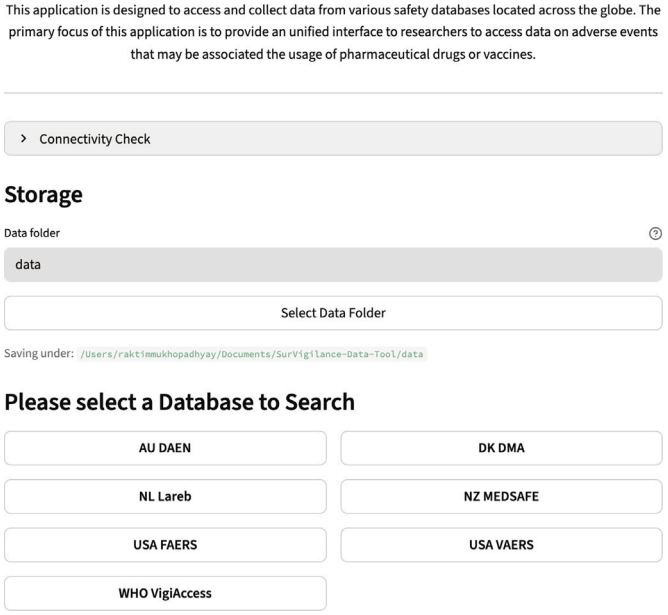

Fig. 3. The home page of SurVigilance displaying the available databases. Please note that the home page contains a button at the top which can be used to specify the location where data will be stored.

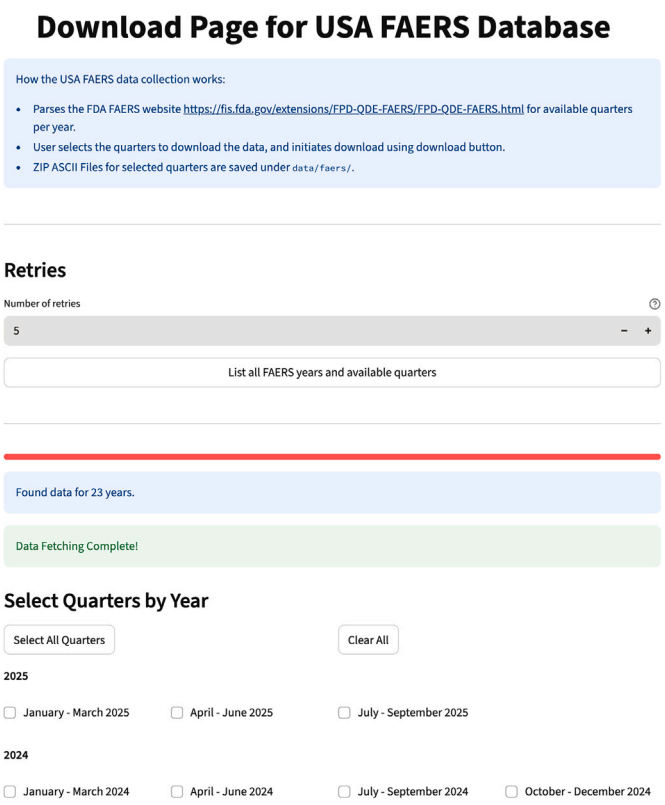

Fig. 4. Data download page for FAERS database. The user can specify the number of retries for listing the available data files on this page. A default value of 5 is set for convenience. After clicking the "List all FAERS years and available quarters" button, all available data files are shown from which a user can select the data to be downloaded.

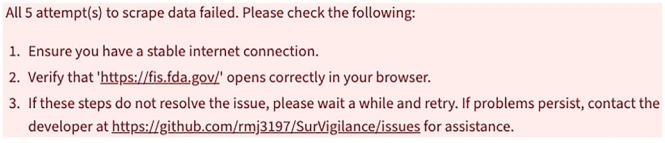

Fig. 5. An informative error message, with troubleshooting details is shown to the user in case any error is encountered with listing the quarters for which data are available. The number of retries were set to 5 as seen in Fig. 4. The GUI provides the user the ability to set any custom value.

FAERS database from which data can be downloaded. In case of an error, an informative error message is also shown as seen in Fig. 5. In this example, we select the January–March 2025 dataset to be downloaded. The user then needs to scroll down and click on the "Download" button. Once the download is complete, a completion dialog is displayed (Fig. 6).

3.2.2. Construction of a contingency table from downloaded FAERS data

The downloaded data can be used to construct contingency tables suited to a user's needs. In this example, we construct a contingency table of six statin drugs, namely Atorvastatin, Fluvastatin, Lovastatin, Pravastatin, Rosuvastatin, and Simvastatin. A small portion of the constructed contingency table is shown in Table 4. The associated code to process the downloaded data to create the contingency table is included in Appendix B.

The authors in [5] introduce an interesting dichotomy for constructing contingency tables corresponding to AE-based and drug-based analysis. The AE-based analysis is particularly suited to the context of clinical trials where the objective is to determine whether a specific AE is reported more commonly with a given drug compared to other AEs

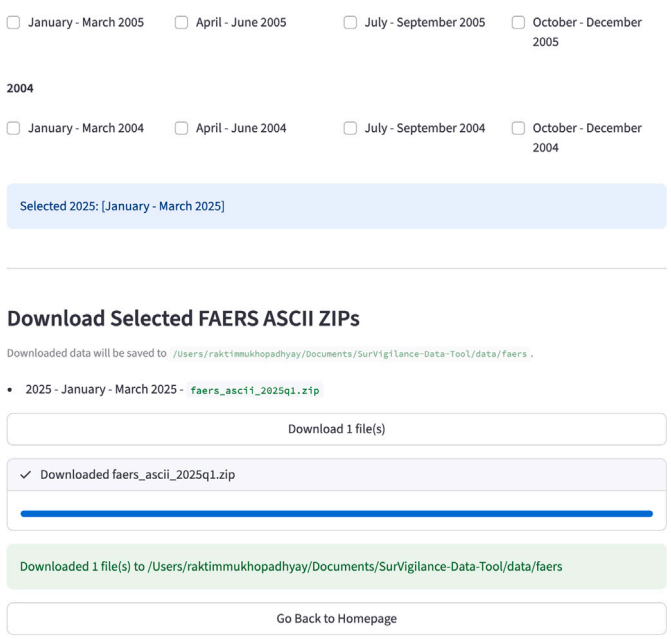

Fig. 6. After selecting the data files to be downloaded, the user needs to scroll to the download button at the bottom of the page. Once the download of the selected files is complete, a success message is shown.

associated with that same drug. For such analysis, the "Other Drugs" column in a 2×2 contingency table includes all reports associated with other drugs excluding the specific drug of interest. In contrast, the drug-based analysis is suitable for post-marketing surveillance where the interest lies in identifying whether a particular drug is associated with abnormally higher frequencies of AEs than other drugs. To avoid confounding from other drugs within the same therapeutic class (e.g., other statin drugs while evaluating atorvastatin), the "Other Drugs" column contains the number of reports associated with all drugs outside that therapeutic class. In Table 4, the "Other Drugs" column contains the number of reports associated with other drugs excluding the statin drug class. This is created with drug-based analysis for post-marketing surveillance.

4. Computational time associated with accessing data

In order to demonstrate the computational efficiency of `SurVigilance` in retrieving data from various safety databases, we perform a small experiment in which data for "Atorvastatin" is retrieved from the five databases, DMA, DAEN, Lareb, Medsafe, and VigiAccess. The data for "Atorvastatin" is retrieved 10 times, using the programmatic access functions included in `SurVigilance` on Python version 3.11.9. The experiment was conducted on an off-the-shelf MacBook Air with an Apple M1 processor running at 3.2 GHz and 16 GB of RAM, using macOS Tahoe. Since the software also needs to access the webpages, internet connectivity is necessary. The experiment was conducted on a 5G Wi-Fi network with a maximum download speed of approximately 450 Mbps. The data retrieval times are shown in Table 5 and the code for running the experiment is included in Appendix C.

From Table 5, we observe that the data retrieval times for the databases are not uniform. In ascending order of data retrieval times (based on the statistics presented in Table 5), the least time is taken to retrieve data from Medsafe, followed by DMA, DAEN, and VigiAccess, while the maximum time is required for Lareb. The data retrieval times depend on a number of aspects. Medsafe displays the data in a tabular format on its webpage without embedding the data inside collapsible elements. This allows the `Python` package `BeautifulSoup` to be used in a

Table 4
A small portion of a contingency table for statin drugs constructed using downloaded data from FAERS. Please note that for constructing the "Other Drugs" column all drugs other than those belonging to the statin drug class have been considered. This is in accordance with the drug-based analysis suitable for a post-marketing surveillance scenario proposed in [5].

PT	Atorvastatin	Fluvastatin	Lovastatin	Pravastatin	Rosuvastatin	Simvastatin	Other drugs
Myalgia	22	0	1	3	24	17	683
Treatment failure	2	0	0	3	3	1	1028
Pain in extremity	7	0	0	2	8	0	913
Suicidal ideation	0	0	0	2	0	0	616

Table 5
Computation time associated with retrieving data for Atorvastatin from five safety databases, DMA, DAEN, Lareb, Medsafe, and VigiAccess.

Database	Data retrieval time (in secs)	
	Mean (SD)	Median $[Q_1, Q_3]$
DMA	29.21 (0.96)	28.98 [28.59, 29.47]
DAEN	40.35 (1.63)	40.18 [39.70, 41.64]
Lareb	129.97 (1.15)	129.62 [129.36, 129.68]
Medsafe	18.40 (0.79)	18.19 [17.81, 18.69]
VigiAccess	57.32 (0.34)	57.25 [57.06, 57.59]

straightforward manner to extract the data. On the other hand, the data in DMA is displayed as a table with expandable elements, where all the elements can be expanded with a single button. Since all elements can be expanded using a single button, the overall parsing process remains relatively fast. In the case of DAEN, most of the time is spent waiting for the webpage to prepare the export file. For VigiAccess, the scraper needs to expand each element one by one to extract the underlying data, which increases the retrieval time. In the case of the Lareb database, the majority of the retrieval time is spent on loading the search results. Similar to VigiAccess, once the search result is displayed, time is spent on expanding elements and extracting the underlying data. Additionally, while developing the scraping functions for the various databases, we have also embedded intentional delays as described in Appendix A. The delays are embedded into various stages of the data retrieval process. The delay durations are not typically the same across the different steps and databases, as the delay durations are also an artifact of a specific database. These delay durations have been set in order to let all elements load in a proper manner on a webpage before progressing to the next step. We have tried to set the delays to an acceptable value which ensures successful scraping and reproducibility in varied conditions (e.g., variable network speed). In certain scenarios, when the delay times were set to lower values, the data retrieval process failed. Hence we have embedded safe delays to maintain the reproducibility of the data retrieval process.

5. Stability assessment

We evaluate the robustness of `SurVigilance` in retrieving data from the various databases by designing an experiment on the Google Cloud Platform (GCP). We execute the codes associated with retrieving data from USA FAERS, AU DAEN, DK DMA, NL Lareb and NZ MedSafe at different times of the day as well as from different geographical locations to understand the temporal as any geographical limitations that might potentially be associated with using the `SurVigilance` tool. A schematic diagram depicting the process is shown in Fig. 7. We containerized the tool (Step 1 of the figure) and deployed it to five geographical locations (Step 2), specifically, asia-east2 (Hong Kong), europe-west8 (Milan), southamerica-east1 (Sao Paulo), us-central1 (Iowa), and africa-south1 (Johannesburg). We executed the data retrieval codes at different times of the day to simulate the data retrieval under varied traffic loads. A table of the specific times when the tool was executed using cron jobs is described in Table 6.

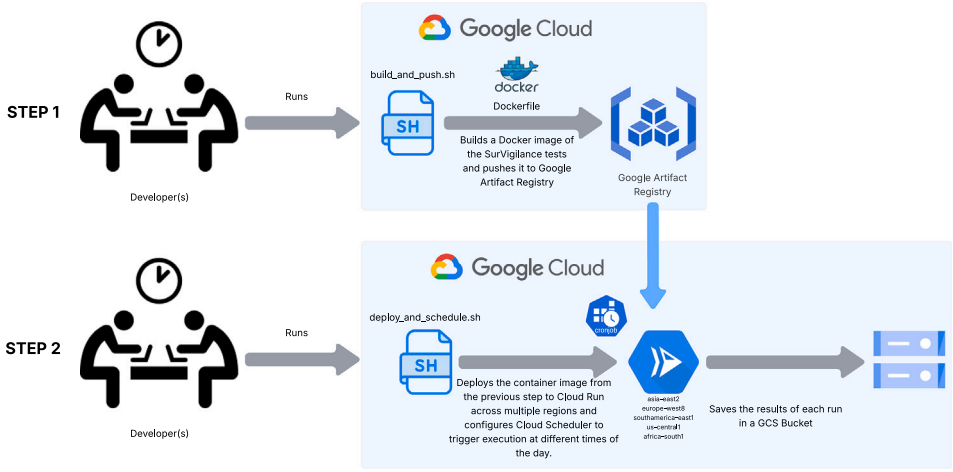

Fig. 7. Schematic diagram of the experiment performed on Google Cloud Platform (GCP) to evaluate the stability of `SurVigilance` in retrieving data. In Step 1, we build the container and upload it to the Google Artifact Registry which makes it available for further tasks on GCP. In Step 2, we deploy the containerized application to different geographical locations and execute at different time of the day using a cron job. The results of the executions are saved in a Google Cloud Storage Bucket. The associated codes can be found at https://github.com/rmj3197/SurVigilance-GCP.

Table 6
Execution times of `SurVigilance` data retrieval codes at the different locations in the experiment.

	Time in UTC	Equivalent time				
		Hong Kong	Milan	Sao Paulo	Iowa	Johannesburg
Late night	1:00 AM	9:00 AM	2:00 AM	10:00 PM	7:00 PM	11:00 PM
	1:15 AM					
	1:30 AM	Increments of 15 minutes				
	1:45 AM					
	2:00 AM	10:00 AM	3:00 AM	11:00 PM	8:00 PM	12:00 AM
Morning	8:00 AM	4:00 PM	9:00 AM	5:00 AM	2:00 AM	6:00 AM
	8:15 AM					
	8:30 AM	Increments of 15 minutes				
	8:45 AM					
	9:00 AM	5:00 PM	10:00 AM	6:00 AM	3:00 AM	7:00 AM
Evening	5:00 PM	1:00 AM	6:00 PM	2:00 PM	11:00 AM	3:00 PM
	5:15 PM					
	5:30 PM	Increments of 15 minutes				
	5:45 PM					
	6:00 PM	2:00 AM	7:00 PM	3:00 PM	12:00 PM	4:00 PM

We executed the tool three times in parallel at each time point. This allowed us to simulate parallel data retrieval as well as distinguish between connectivity issues identified by isolated failures or more involved issues, such as high latency encountered in accessing the databases from a particular region. We furthermore, save the results of the execution in a Google Cloud Storage (GCS) bucket for post processing and summarization. This experiment is an attempt to evaluate the stability of the tool under varied conditions to retrieve the data. The code associated with the experiment can be found at https://github.com/rmj3197/SurVigilance-GCP. The results of this experiment are summarized in Table 7. From Table 7, we observe that `SurVigilance` does not produce an error in the case of any database except for NZ SMARS. The errors in the case of NZ SMARS are intermittent and not limited to any particular time of the day or geographical location. The webpage specifically uses the Microsoft.NET Framework and ASP.NET (Active Server Pages) in order to display the data from the underlying database. We encountered intermittent errors such as shown in Fig. 8 which resolved spontaneously after a while. However, these temporary errors cause the web scraping process to fail. Since this is an issue on the server side, we are unable to prevent this error. We suggest using a higher number of retries and waiting for a specific time interval if a user encounters errors with NZ SMARS.

From the above discussion it is evident that, `SurVigilance` is a robust tool capable of retrieving data even when used from various parts of the world for multiple databases.

6. Limitations

`SurVigilance` provides access to data from various safety databases, which primarily provide access to the data through respective webpages. In order for the tool to be able to retrieve data from those webpages, we also need to take into account updates or changes to the structure of a webpage, and implement any additional measures for continued access to the data. `SurVigilance` might need to be regularly updated to keep it functional, otherwise the tool risks becoming outdated. This challenge exists due to the ephemeral nature of webpages. We have deployed an automated workflow using `GitHub Actions` that runs the test suite every day. This workflow is designed to fail and trigger notifications to the developer if any of the tests fail, thus allowing us to continuously monitor updates to the database webpages. We also maintain an automated daily status page at https://survigilance.netlify.app/daily_status which reflects whether the tool was able to retrieve data from the databases or encountered any issues. Additionally, certain databases such as `VAERS` implement CAPTCHAs, and hence are not completely automatable without human intervention.

7. Impact

`SurVigilance` aims to streamline the process of accessing pharmacovigilance data. The tool greatly reduces the time and complexity associated with accessing data from safety databases across the world. By providing both graphical and programmatic interfaces to the underlying methods, `SurVigilance` is appropriately suited for integration into pre-existing analytical pipelines. `SurVigilance` has been designed with the maintainability and extensibility of the codebase in mind for the long term, supporting additional databases in the future. The tool furthermore gives users access to the data in raw form, without any additional processing that researchers might find unnecessary or unsuited to their needs. A search on Google Scholar for the term “pharmacovigilance” returns approximately 25,000 search results since 2021, highlighting the extent of research in the domain. We hypothesize that the tool will be of practical importance to researchers in this domain. `SurVigilance` thus addresses a key methodological barrier in the domain by streamlining data acquisition while leaving analytical choices to the investigator.

Table 7
Summarized results of the experiment conducted on GCP highlighting that `SurVigilance` facilitates a robust data retrieval process for all databases with the exception of NZ SMARS in which temporary errors were observed. Even in scenarios where errors were encountered for NZ SMARS, the errors resolved within the next 15 minute interval. The raw data files and the data processing scripts used to generate this table are available at https://github.com/rmj3197/SurVigilance-GCP.

Time UTC	Johannesburg	Hong Kong	Milan	Sao Paulo	Iowa
1:00:00	✓	✓	✓	NZ SMARS Failed 1/3 Times	✓
1:15:00	✓	✓	✓	✓	✓
1:30:00	✓	✓	✓	✓	✓
1:45:00	✓	✓	✓	✓	✓
2:00:00	✓	✓	NZ SMARS Failed 1/3 times	✓	✓
8:00:00	NZ SMARS failed 2/3 times	NZ SMARS failed 2/3 Times	✓	✓	✓
8:15:00	✓	✓	✓	✓	✓
8:30:00	✓	✓	✓	✓	✓
8:45:00	✓	✓	✓	✓	✓
9:00:00	✓	✓	✓	✓	✓
17:00:00	✓	✓	✓	✓	✓
17:15:00	✓	✓	✓	✓	✓
17:30:00	✓	✓	✓	✓	✓
17:45:00	✓	✓	✓	✓	NZ SMARS failed 1/3 times
18:00:00	✓	✓	✓	✓	✓

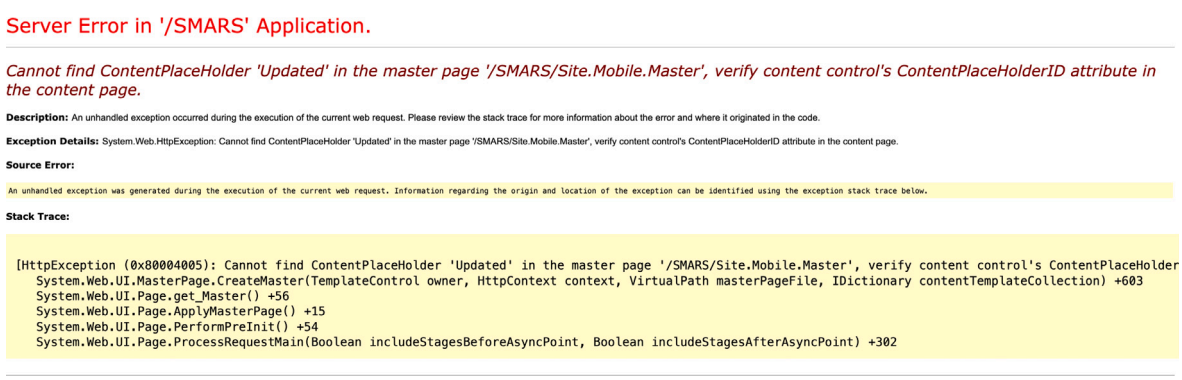

Fig. 8. Intermittent error message associated with NZ SMARS. This error usually resolves spontaneously, and data can be retrieved after a wait period.

8. Conclusions

In this work, we introduce `SurVigilance`, a modular, open-source tool that simplifies access to pharmacovigilance data from multiple global safety databases. A comparison with other similar software packages is included in Table 2, which illustrates that most software solutions focus on a single database (FAERS) while other packages focus on providing helper functions for data processing from FAERS or VigiAccess. A number of packages also provide snapshots of the VAERS database. We demonstrate how our tool can be used to download data using the graphical interface and subsequently process it to create contingency tables used in various methods in pharmacovigilance research. In addition to the graphical interface, `SurVigilance` also offers programmatic access to the databases for more advanced users, with reasonable data retrieval times. While the tool currently supports seven databases, the modular structure of the codebase allows for further extension of the software in the future. In conclusion, `SurVigilance` aims to further global pharmacovigilance research by removing barriers to accessing safety data.

CRediT authorship contribution statement

Raktim Mukhopadhyay: Writing – original draft, Writing – review & editing, Validation, Software, Investigation. **Marianthi Markatou:** Conceptualization, Validation, Investigation, Resources, Writing – original draft, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We gratefully acknowledge the organizations that maintain the various safety databases included in our work, whose efforts make these datasets publicly accessible. The senior author acknowledges financial support in the form of a research award from KALEIDA Health Foundation, USA (Grant Number 82114), which supported the work of the first author.

Appendix A. Responsible use

We extensively referred to the Responsible Use guidance provided as part of the publication [33], which is available at https://github.com/truongbavinh/HTMLDownloader-/blob/master/docs/LEGAL.md.

We have undertaken the following measures to promote responsible use:

1. Adhering to the guidelines mentioned above, we have not implemented any concurrency or parallelism to prevent overwhelming the servers with traffic while retrieving the data.
2. We have embedded delays while scraping data to further prevent stressing the server and to emulate human-like access patterns. This can be clearly seen from the data retrieval times illustrated in Table 5.
3. We did not attempt to circumvent additional security measures such as CAPTCHAs which are present for accessing the VAERS database.
4. We verify the presence of `robots.txt` files and ensure that the scraping of the database is not prohibited. The URLs to access the `robots.txt` for the various databases are shown in Table A.8.
5. The terms of use associated with the various webpages to access the databases can be found in Table A.9. To the best of our knowledge, these webpages do not explicitly prohibit web scraping.

Table A.8
URLs associated with the `robots.txt` files for various databases.

Database	`robots.txt` Present	Link
DAEN	Yes	https://www.tga.gov.au/robots.txt
DMA	No	NA
FAERS	Yes	https://www.fda.gov/robots.txt
Lareb	No	NA
Medsafe	Yes	https://www.medsafe.govt.nz/robots.txt
VAERS	No	NA
VigiAccess	No	NA

Table A.9
URLs for the "Terms of Use/Service" associated with the webpages used to access the databases.

Database	Link for terms of Use
DAEN	https://www.tga.gov.au/about-tga/using-our-website/copyright
DMA	https://laegemiddelstyrelsen.dk/en/sideeffects/side-effects-of-medicines/interactive-adverse-drug-reaction-overviews/terms-of-use-of-the-interactive-adr-overviews/
FAERS	https://www.fda.gov/about-fda/about-website/website-policies
Lareb	https://www.lareb.nl/en/pages/privacy-statement/
Medsafe	https://www.medsafe.govt.nz/other/siteinfo.asp
VAERS	https://vaers.hhs.gov/privacy.html
VigiAccess	https://www.vigiaccess.org/

Appendix B. Code for processing downloaded FAERS data

```python
import pandas as pd
from pathlib import Path
import zipfile

# Unzip once into a local folder
zip_path = Path(
    "data/faers/faers_ascii_2025q1.zip"
)
out_dir = Path("extracted_files")
out_dir.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(zip_path, "r") as zf:
    zf.extractall(out_dir)

# Get the paths for the DRUG* and REAC* files
drug_path = next(out_dir.rglob("ASCII/DRUG*.txt"))
reac_path = next(out_dir.rglob("ASCII/REAC*.txt"))

# Read both files ($ delimited)
read_options = dict(sep=r"\$", engine="python", dtype=str)
drug = pd.read_csv(drug_path, **read_options)
reac = pd.read_csv(reac_path, **read_options)

# Keep just the columns we care about from DRUG and filter to PS/SS
drug = drug.rename(columns=str.lower)[
    ["primaryid", "caseid", "drug_seq", "role_cod", "prod_ai"]
]
reac = reac.rename(columns=str.lower)
drug = drug[drug["role_cod"].isin(["PS", "SS"])]

# Join DRUG and REAC on primaryid + caseid
dat = pd.merge(drug, reac, on=["primaryid", "caseid"], how="outer")

# For each caseid, keep the row with the largest primaryid (treat that as "most recent")
dat["_pid_num"] = pd.to_numeric(dat["primaryid"], errors="coerce")
dat = (
    dat.sort_values(["caseid", "_pid_num"])
    .groupby("caseid", as_index=False)
    .tail(1)
    .drop(columns="_pid_num")
)


# Make statins in ALL CAPS; everything else becomes OTHER
def label_statin(s):
    s = (s or "").lower()
    for d in [
        "atorvastatin",
        "fluvastatin",
        "lovastatin",
        "pravastatin",
        "rosuvastatin",
        "simvastatin",
    ]:
        if d in s:
            return d.upper()
    return "OTHER"


dat["DRUG_NEW"] = dat["prod_ai"].apply(label_statin)

# Keep only PTs that actually co-occur with a statin
statin_pts = dat.loc[dat["DRUG_NEW"] != "OTHER", "pt"].dropna().unique()
dat = dat[dat["pt"].isin(statin_pts)]

# Build the contingency table
pt_table = (
    pd.pivot_table(
        dat,
        index="pt",
        columns="DRUG_NEW",
        values="primaryid",
        aggfunc="count",
        fill_value=0,
    )
    .reset_index()
    .rename(columns={"pt": "PT"})
)
pt_table.columns = [c.upper() for c in pt_table.columns]
pt_table = pt_table[
    [c for c in pt_table.columns if c != "OTHER"]
    + (["OTHER"] if "OTHER" in pt_table.columns else [])
]

# Save the file
pt_table.to_excel("STATIN_ALL_PT.xlsx", index=False)
```

Listing 3. Python code for processing the downloaded FAERS data to generate a contingency table for statin drugs.

Appendix C. Code to measure data retrieval time for atorvastatin

```python
import sys
import time
import pandas as pd

# settings
RUNS = 10
DRUG = "atorvastatin"
HEADLESS = True

from SurVigilance.ui.scrapers import (
    scrape_daen_sb,
    scrape_dma_sb,
    scrape_lareb_sb,
    scrape_medsafe_sb,
    scrape_vigiaccess_sb,
)

def time_call(fn):
    start = time.perf_counter()
    fn()
    return time.perf_counter() - start

def main():
    targets = {
        "DMA": lambda: scrape_dma_sb(medicine=DRUG, headless=HEADLESS),
        "DAEN": lambda: scrape_daen_sb(medicine=DRUG, headless=HEADLESS),
        "Lareb": lambda: scrape_lareb_sb(medicine=DRUG, headless=HEADLESS),
        "Medsafe": lambda: scrape_medsafe_sb(
            searching_for="medicine", drug_vaccine=DRUG, headless=HEADLESS
        ),
        "VigiAccess": lambda: scrape_vigiaccess_sb(medicine=DRUG, headless=HEADLESS),
    }

    print(f"Recording time for '{DRUG}' runs per database={RUNS}\n")

    results = {name: [] for name in targets}

    # round-robin execution
    for r in range(1, RUNS + 1):
        for name, fn in targets.items():
            dur = time_call(fn)
            results[name].append(dur)
            print(f"  {name:10s} run {r:02d}: {dur:.2f}s")

    print("\n Descriptive Summary")

    summary_data = []
    for name in targets:
        s = pd.Series(results[name])
        mean = s.mean()
        sd = s.std()
        median = s.median()
        q1 = s.quantile(0.25)
        q3 = s.quantile(0.75)

        summary_data.append({
            "Database": name,
            "Mean (SD)": f"{mean:.2f} ({sd:.2f})",
            "Median [Q1, Q3]": f"{median:.2f} [{q1:.2f}, {q3:.2f}]"
        })

        print(
            f"{name}  n={len(s)}  mean={mean:.2f}s  median={median:.2f}s  sd={sd:.2f}s"
        )
```

```
    df_summary = pd.DataFrame(summary_data)
    output_file = f"scraper_timing_summary_{DRUG}.csv"
    df_summary.to_csv(output_file, index=False)

if __name__ == "__main__":
    main()
```

Listing 4. `Python` code to record data retrieval times for Atorvastatin from multiple safety databases.

Data availability

All data shown in this work are publicly available from the respective safety databases. The data for Example 3.1 can be accessed from https://laegemiddelstyrelsen.dk/sideeffects/side-effects-of-medicines/interactive-adverse-drug-reaction-overviews/.aspx and the data for Example 3.2 can be found at https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html. The associated `Python` scripts to download and process the data are also included as part of this work.

References

[1] World Health Organization. Patient safety, 2023. https://www.who.int/news-room/fact-sheets/detail/patient-safety [Online accessed 26 Sep 2025].

[2] White TJ, Arakelian A, Rho JP. Counting the costs of drug-related adverse events. Pharmacoeconomics 1999;15(5):445–58.

[3] Food and Drug Administration, Preventable Adverse Drug Reactions: A Focus on Drug Interactions. 2018. https://www.fda.gov/drugs/drug-interactions-labeling/preventable-adverse-drug-reactions-focus-drug-interactions [Online accessed 26 Sep 2025].

[4] Amery WK. ISPE, why there is a need for pharmacovigilance, pharmacoepidemiol. Drug Saf 1999;8(1):61–4.

[5] Ding Y, Markatou M, Ball R. An evaluation of statistical approaches to postmarketing surveillance. Stat Med 2020;39(7):845–74.

[6] Huang L, Guo T, Zalkikar JN, Tiwari RC. A review of statistical methods for safety surveillance. Ther Innov Regul Sci 2014;48(1):98–108.

[7] Chakraborty S, Liu A, Ball R, Markatou M. On the use of the likelihood ratio test methodology in pharmacovigilance. Stat Med 2022;41(27):5395–420.

[8] Kulldorff M, Fang Z, Walsh SJ. A tree-based scan statistic for database disease surveillance. Biometrics 2003;59(2):323–31. https://doi.org/10.1111/1541-0420.00039

[9] Fouretier A, Malriq A, Bertram D. Open access pharmacovigilance databases: analysis of 11 databases. Pharmaceut Med 2016;30(4):221–31. https://doi.org/10.1007/s40290-016-0146-6

[10] Therapeutic Goods Administration, Database of Adverse Event Notifications (DAEN). 2025. https://www.tga.gov.au/safety/safety/safety-monitoring-daen-database-adverse-event-notifications/database-adverse-event-notifications-daen [Online accessed 9 Jan 2025].

[11] Danish Medicines Agency, Interactive Adverse Drug Reaction Overviews. 2025. https://laegemiddelstyrelsen.dk/en/sideeffects/side-effects-of-medicines/interactive-adverse-drug-reaction-overviews/ [Online accessed 29 Sep 2025].

[12] Van Puijenbroek E, Van Grootheest K. Organization of pharmacovigilance in the Netherlands, vol. 13: John Wiley & Sons, Ltd; 2014. pp. 213–6. https://doi.org/10.1002/9781118820186.ch13e

[13] Netherlands Pharmacovigilance Centre Lareb, About lareb. 2025. https://www.lareb.nl/en/pages/about-lareb [Online accessed 9 Jan 2025].

[14] Food and Drug Administration. FDA's adverse event reporting system (FAERS), 2024. https://www.fda.gov/drugs/surveillance/fdas-adverse-event-reporting-system-faers [Online accessed 29 Sep 2025].

[15] Centers for Disease Control and Prevention, Vaccine Adverse Event Reporting System (VAERS). https://vaers.hhs.gov/about.html [Online accessed 9 Jan 2025].

[16] Uppsala Monitoring Centre, The global medicines safety database. 2025. https://who-umc.org/vigibase/ [Online accessed 9 Jan 2025].

[17] Shankar PR. VigiAccess: promoting public access to VigiBase. Indian J Pharmacol 2016;48(5):606–7.

[18] Mintz M. SeleniumBase DOCS, 2024. https://seleniumbase.io/ [Online accessed 30 Sep 2025].

[19] World Wide Web Consortium (W3C). Webdriver, 2025. https://www.w3.org/TR/webdriver2/ [Online; accessed 30 Sep 2025].

[20] Yeen J, Sadym M. A look back in time: the evolution of test Automation, 2023. https://developer.chrome.com/blog/test-automation-evolution#webdriver_classic_versus_chrome_devtools_protocol_cdp [Online accessed 2 Oct 2025].

[21] Peng Y, Song Y, Qin C, Lin J. Faers: r interface for FDA adverse event reporting system, r package version 1.4.0, 2025. https://doi.org/10.18129/B9.bioc.faers. https://bioconductor.org/packages/faers.

[22] Yang R. extractFAERS: extract data from FAERS database, R package version 0.1.4, 2025. https://CRAN.R-project.org/package=extractFAERS.

[23] Garcez L. Faersquarterlydata: FDA adverse event reporting system quarterly data extracting tool, r package version 1.2.0, 2024. https://CRAN.R-project.org/package=faersquarterlydata.

[24] Bernauer ML. FAERS: repository for downloading, processing, and analyzing FDA adverse event reporting system data, 2018. https://github.com/mlbernauer/FAERS [Online accessed 14 Sep 2025].

[25] Böhm R. OpenVigil - open tools for data-mining and analysis of pharmacovigilance data, 2025. https://openvigil.sourceforge.net/ [Online accessed 14 Sep 2025].

[26] Böhm R, von Hehn L, Herdegen T, Klein H-J, Bruhn O, Petri H, Höcker J. Openvigil fda–inspection of us American adverse drug events pharmacovigilance data and novel clinical applications. PloS One 2016;11(6):e0157753.

[27] Embry I, vaers, 2021. https://gitlab.com/iembry/vaers [Online accessed 14 Sep 2025].

[28] Embry I. Vaersvax: US vaccine adverse event reporting system (VAERS) vaccine data for present, r package version 1.0.5, 2018. https://CRAN.R-project.org/package=vaersvax.

[29] Embry I. vaersNDvax: Non-Domestic vaccine adverse event reporting system (VAERS) vaccine data for present, r package version 1.0.4, 2016. https://CRAN.R-project.org/package=vaersNDvax.

[30] Dolladille C, Chrétien B. Vigicaen: 'VigiBase' pharmacovigilance database toolbox, r package version 0.16.1, 2025. https://CRAN.R-project.org/package=vigicaen.

[31] Liu A, Mukhopadhyay R, Markatou M. MDDC: an r and Python package for adverse event identification in pharmacovigilance data. Sci Rep 2025;15(1):21317.

[32] Medical Dictionary for Regulatory Activities (MedDRA), MedDRA Hierarchy, 2025https://www.meddra.org/how-to-use/basics/hierarchy [Online accessed 30 Sep 2025].

[33] Truong B-V, Nguyen LT, Pham P, Vo B. HTMLDownloader: an open-source tool for dynamic web scraping and archiving using webview2. SoftwareX 2025;32:102373. https://doi.org/10.1016/j.softx.2025.102373. https://www.sciencedirect.com/science/article/pii/S2352711025003395.